Full Web and Mobile Application Development Assignment

Choose one application from Musasizi's public GitHub repositories, run its frontend and backend locally, and add at least three new features. Demonstrate that you understand the existing code and can extend it into a working application.

Learning outcomes

- Configure an existing frontend, API and database to work together.
- Trace a user action through the interface, API and database.
- Design, implement and test useful features while preserving existing workflows.
- Use Git to record changes and explain your implementation clearly.

Choose one application

Fork both repositories in one matching pair into your own GitHub account. Clone your forks locally and retain the original project attribution. Repository names below are clickable.

Application	Frontend repository	Backend repository
CleanTrack	cleantrack-frontend	cleantrack-backend
JobConnect	jobconnect-frontend	jobconnect-backend
FarmConnect	farmconnect-frontend	farmconnect-backend
LearnHub	learnhub-frontend	learnhub-backend

To use another application from the same GitHub account, obtain the lecturer's approval of the matching repository pair before starting.

Set up and verify the original application

Follow the README in each selected repository. Setup commands and database scripts vary between projects; use the instructions for your chosen pair.

1. Install compatible Node.js and npm versions, plus MySQL or MariaDB. Check package requirements as well as the README.
2. Install frontend and backend dependencies. Create local environment files from the supplied examples and configure database credentials, API URLs and allowed frontend origins.
3. Create a separate local learning database and load the supplied schema and seed data. Check scripts before running them because some recreate the database.
4. Start the backend and frontend in separate terminals. Confirm that the interface loads data from your local API.
5. Demonstrate login and one complete existing workflow, including a database change where applicable. Record the original commit ID of each repository before making changes.
6. Document setup problems and how you resolved them. Keep secrets and local environment files out of Git; provide example configuration with placeholder values.

Propose and implement three new features

Submit a short proposal for review before substantial implementation. For each feature, state the user, the problem it solves, the expected behaviour, the frontend and backend changes, any database changes, and clear acceptance criteria.

- Implement at least three distinct, useful features beyond what exists in your recorded starting version.
- At least two features must include both frontend and backend changes. Persist data in the database whenever the feature requires saved records.
- One feature means one complete user capability. Its interface, endpoint and database table do not count as three separate features.
- Colour changes, button renaming, dependency updates and setup fixes do not count as new features. Existing functionality also does not count.
- Include suitable validation, loading and error feedback, and role or ownership checks where relevant. Existing core workflows must continue to work.

Possible feature ideas

Use these as starting points, then check the current application and define a proposal that adds new behaviour. They are suggestions, not mandatory features.

- CleanTrack: recurring pickup schedules with an interface and saved schedule records.
- JobConnect: saved job alerts with preferences and an in-app matching list.
- FarmConnect: database-backed favourites that remain available after login on another browser. The existing frontend documents locally saved favourites.
- LearnHub: a discussion area attached to each course with appropriate participant access.

Test and prepare your submission

Test all three features and check the original login and core workflow again. Record the steps, expected result, actual result and pass or fail outcome. Include successful use, invalid input, empty states, and unauthorised access where relevant. Use automated tests where practical; reproducible manual tests are acceptable.

Submission checklist

- Links to your frontend and backend forks, with final commit IDs so the assessed version is identifiable.
- Updated README instructions sufficient for another person to set up both applications, including prerequisites, configuration, database setup and startup commands.
- Database migration or setup scripts and sample data needed by your new features, where applicable. Do not submit only a database stored on your computer.
- A short report with your name and student number, chosen application, original commit IDs, setup issues, and a summary of each feature and its acceptance criteria.
- Screenshots or a short screen recording showing the original working application and all three new features, plus your testing results and known limitations.
- Acknowledgement of external code, libraries, references and AI assistance used, where applicable. You must be able to explain and modify the code you submit.

Keep generated dependencies, build output, real credentials and private user data out of your submission. Use meaningful commits that show your implementation progress. Submit through the channel announced by the lecturer.

Demonstration

Be ready to run the application, demonstrate each feature against its acceptance criteria, and explain one request from the interface through the API to the database. The lecturer may ask you to explain access checks or make a small change to confirm your understanding.

Marking rubric

Criterion	Assessment focus	Marks
Local setup	Frontend, backend and database work together; original login and workflow demonstrated.	20
Three new features	15 per feature: usefulness and scope 3; working implementation 8; validation and relevant access checks 4.	45
Code and integration	Readable code, consistent project structure, reliable API integration and appropriate database changes.	15
Testing and documentation	Reproducible setup instructions, clear feature descriptions, and meaningful testing evidence.	10
Demonstration	Working feature demonstrations and understanding of the submitted code and data flow.	10
Total		**100**

Marks reward working, demonstrable functionality. Document incomplete features and limitations honestly; a feature description alone does not demonstrate implementation.